Reporte Técnico POO

CLASES MODELO

FRANCISCO ALEJANDRO RAMIREZ GARCIA

PROGRAMACION ORIENTADA A OBJETOS 3-A EVND

Proceso de Abstracción Aplicado al Proyecto

Para aplicar el concepto de abstracción en el sistema del Taller Mecánico, analicé qué información de las personas era realmente importante guardar en el programa y cuál no hacía falta. En la vida real, una persona tiene muchos datos como su estatura o pasatiempos, pero para el sistema del taller solo necesitamos la información básica para identificarla y darle acceso al programa.

Por esta razón, dividí la estructura en dos clases organizadas según su uso:

Resumen del Proceso de Abstracción:

Clase General (Persona): Contiene los datos personales que cualquier ser humano dentro del sistema tendría: su ID único, CURP, RFC, nombre, apellido, fecha de nacimiento, correo, teléfono y la foto de perfil.

Clase Específica (Administrador): Guarda solo los datos adicionales que necesita un administrador para entrar y usar el sistema, como su usuario, contraseña, estado en el sistema (ACTIVO o INACTIVO) y su rol.

Encapsulamiento y Ocultación de Información

El encapsulamiento se aplicó en todas las clases poniendo todos los atributos con el modificador private. Esto evita que desde otras partes del código (como la ventana o los controladores) se puedan cambiar las variables directamente sin control.

Para leer o modificar estos valores, creé los métodos públicos getters y setters. Dentro de los setters agregué validaciones sencillas para asegurarme de que los datos introducidos sean correctos.

Cómo se Aplicó el Encapsulamiento en Cada Clase:

En Persona: En el método setId(int id) reviso que el número no sea negativo. En setNombre() valido que el nombre no venga vacío o solo con espacios. Además, agregué métodos como getNombres() y setNombres() para que la tabla de JavaFX pueda leer los datos sin problemas.

En Administrador: En el método setUsuario() reviso que el usuario no esté vacío y controlo el estado usando el enum EstadoAdministrador para evitar que pongan un estado inventado.

Justificación de la Herencia, Clase Abstracta y Palabra final

Definí la clase Persona como abstract porque en el taller mecánico no existen personas "en general", sino tipos de personas concretas como administradores o clientes. Al ponerla como abstracta, evito que alguien cree un objeto directo con new Persona(), obligando a usar una clase derivada.

Por otro lado, le puse la palabra clave final a la clase Administrador porque es el último nivel de esa rama. No hace falta que nadie vuelva a heredar de Administrador, por lo que declarar la clase como final evita herencias innecesarias y protege el código.

Análsis de Cohesión y Acoplamiento

En el diseño de las clases busqué mantener una buena organización mediante estos dos principios:

Alta Cohesión: Cada clase se encarga únicamente de lo que le corresponde. La clase Persona solo maneja datos personales generales y la clase Administrador solo maneja los datos de acceso al programa. Ninguna de las dos tiene código de la interfaz gráfica o de botones.

Bajo Acoplamiento: Para comparar si dos personas son iguales en los métodos equals() y hashCode(), utilicé únicamente el atributo id. En la clase Administrador, usé super.equals() y super.hashCode() para reutilizar lo que ya hace la clase base. De la misma forma, el método toString() de Administrador combina super.toString() con sus datos propios, logrando un código fácil de mantener y entender.